

Compilers tutorial IV: Abstract syntax

Yacc is a powerful tool for generating parsers, by transforming formal grammars into executable code. In this tutorial we explore how to construct an abstract syntax tree (AST), which is a kind of parse tree which discards irrelevant details while fully preserving the meaning of the original program.

A bit of theory: A syntax-directed translation (SDT) scheme consists of a grammar with attached program fragments (called *actions*). Whenever a *production* is used, during syntax analysis, its *action* is executed. One of the main uses of these schemes is to build syntax trees. Every time a production is used during bottom-up parsing, the corresponding action can *create* new nodes and/or *relate* children nodes to the parent.

Abstract syntax trees

Trees can be represented in a number of ways. The data structures below specify that a *node* in the AST has a linked list of *children nodes*:

```
struct node {
    enum category category;
    char *token;
    struct node_list *children;
};

struct node_list {
    struct node *node;
    struct node_list *next;
};
```

Every node has a syntactic *category* denoting a specific programming construct occurring in the input program, such as a function, a parameter declaration, or a natural constant:

```
enum category {Program, Function, ..., Identifier, Natural, ...};
```

Every node also includes a pointer to the original *token*. The token is necessary because a node of category **Identifier** must have the name of the function or variable, a node of category **Natural** must have the string of digits representing the natural constant, and so on.

To build an AST we only need two operations: creating a new node and adding a child node to a parent node. Two functions provide that functionality:

```
struct node *newnode(enum category category, char *token);
void addchild(struct node *parent, struct node *child);
```

The first function returns a newly allocated node with all its fields initialized (including an empty list of children). The second function appends a node to the list of children of the parent node. They are provided in files `ast.h` and `ast.c`.

Syntax-directed translation

During bottom-up parsing, an action $\{\dots\}$ is executed when the corresponding production is used. The combined result of all those executions is the AST.

Consider the following *yacc* specification (which you will complete). Notice that it is included in file `petit.y`, that you should carefully analyze.

```
program: function          { ast = $$ = newnode(Program, NULL);
                           addchild($$, $1); }

;

function: IDENTIFIER '(' parameters ')' '=' expression
        { $$ = newnode(Function, NULL);
          addchild($$, newnode(Identifier, $1));
          addchild($$, $3);
          addchild($$, $6); }

;

parameters: parameter      { /* ... */ }
           | parameters ',' parameter { /* ... */ }

;

parameter: INTEGER IDENTIFIER { /* ... */ }
          | DOUBLE IDENTIFIER { /* ... */ }

;

arguments: expression      { /* ... */ }
          | arguments ',' expression { /* ... */ }

;

expression: IDENTIFIER      { /* ... */ }
           | NATURAL        { $$ = newnode(Natural, $1); }
           | DECIMAL        { /* ... */ }
           | IDENTIFIER '(' arguments ')' { /* ... */ }
           | IF expression THEN expression ELSE expression %prec LOW
           { /* ... */ }
           | expression '+' expression { /* ... */ }
           | expression '-' expression { /* ... */ }
           | expression '*' expression { /* ... */ }
           | expression '/' expression { /* ... */ }
           | '(' expression ')'      { $$ = $2; }

;
```

When the production for `function` is used, the right-hand side contains grammar symbols representing the function's components. The action executes four statements: (1) a new `Function` node is created; (2) a new `Identifier` node is created, holding the function name, and becomes a child of the `Function` node; (3) the `parameters` node `$3` becomes a child of the `Function` node; and (4) the `expression` node `$6` becomes a child of the `Function` node. Then, parsing will complete at the rule for `program`, where the AST's root node is created with a child function (`ast` is declared as a global variable).

Token types and %union

As seen in the previous tutorial, the semantic value of a token must be stored in global variable `yylval`, which has type `int` by default. Token declarations use:

```
%token INTEGER DOUBLE IF THEN ELSE
```

When we need to use multiple data types, the `%union` declaration allows us to specify the distinct types that might be stored in `yylval`. In our case:

```
%union{
    char *lexeme;
    struct node *node;
}
```

This `%union` declaration modifies the type of `yylval` so that it may hold a *lexeme* (`char *`) or a *node* (`struct node *`). In other words, `yylval` might be a string or an AST node, and a C union encapsulates the two alternatives.

Then, when we declare a token, the C type is specified as follows:

```
%token<lexeme> IDENTIFIER NATURAL DECIMAL
```

Identifiers, naturals and decimals require their semantic value (the `char *` to their string) to be stored. The *lex* specification should copy their value by executing `yylval.lexeme = strdup(yytext);` before returning any such tokens.

Syntactic variables (i.e., nonterminals) are specified using the `%type` declaration:

```
%type<node> program parameters parameter arguments expression
```

This way, the type of `$$`, `$1`, `$2`, etc., is correctly handled during parsing.

Exercises

Begin by carefully examining the file `petit.y` and the AST construction functions in files `ast.h` and `ast.c`.

1. Complete the actions marked with `/* ... */` so that an AST is constructed, for each program, using the supplied functions `newnode(...)` and `addchild(...)`.
2. Write a function to recursively traverse the AST and show its content. The goal is to call that function immediately after `yyparse()` to check that the AST is correct. Consider the following pseudocode:

```
show(struct node *node, int depth) {
    print(node->category, node->token)
    foreach child in node->children show(child, depth+1)
}
```

Taking `factorial(integer n) = if n then n * factorial(n-1) else 1` as input, the solution to exercises 1 and 2 should have the output that follows.

```

Program
__Function
----Identifier(factorial)
----Parameters
-----Parameter
-----Integer
-----Identifier(n)
----If
----Identifier(n)
----Mul
----Identifier(n)
----Call
-----Identifier(factorial)
-----Arguments
-----Sub
-----Identifier(n)
-----Natural(1)
----Natural(1)

```

3. Modify the grammar to allow for multiple functions, using these rules:

```

program: functions
      ;
functions: function
         | functions function
      ;

```

In this case, we need to create a *list* of **Function** nodes. There are many solutions. First, we can create an **Auxiliary** node (a new category) to hold the intended list. We can later assign `$1->category = Program` to change its category. Second, declaring `%type<node_list> functions` makes it explicitly a node list. We will need the list operations `newlist`, `append` and `addchildren`.

Test your solution with the following example, found in file `factorial.pt`:

```

factorial(integer n) = if n then n * factorial(n-1) else 1
main(integer i) = write(factorial(read(0)))

```

The file `factorial.ast` contains the expected AST for this program.

Author

Raul Barbosa (University of Coimbra)

References

Barbosa, R. (2025). Petit programming language and compiler.
<https://github.com/rbbarbosa/Petit>